

Please view 2a, 2b first, output of which were used to summarize using TL;DR and then ELI-15 for 1a and 1b.

Contents

2. Use structured reasoning / step-by-step prompting to generate:	2
a) Automation Framework Architecture	2
Used Chat GPT.....	2
Prompt:	2
Question 1 of 5.....	3
Question 2 of 5.....	4
Question 3 of 5.....	4
Question 4 of 5.....	4
Question 5 of 5.....	5
===== Output=====	5
b) Test Strategy Document.....	14
Prompt:	14
===== Output after question answers ===	17
1a) TL;DR	27
Gave input	27
Observations	27
output.....	27
1b) ELI-15 Explain as if 15 years old is reading.	33
Input: ELI-15	33
Observations:	33
Output:	33

2. Use structured reasoning / step-by-step prompting to generate:

a) Automation Framework Architecture

Used Chat GPT

Prompt:

Instructions:

Generate architecture for automation framework based on Selenium java based with Rest Assured for API testing.

Architecture should document the motivation to do automation and how architecture will achieve it.

[MANDATORILY] Ask 5 questions one by one of multiple choice to take inputs for framework. After taking answer proceed to next question.

[STRICTLY] Once input to all questions are available, proceed to generate the framework structure.

[STRICTLY] proceed step by step for framework implementation.

Context:

Large enterprise needs a scalable, maintainable, extensible automation framework

The framework should support being run from Jenkins

The utility functions to open data files like excel should be in utility folder

Page object model should be used

Cucumber BDD should be supported

Should be using Maven

the data files should be in separate folder

the excel should have IDs for each dataset that will be referred into test case.

Examples:

data excel ==> should have tabs like "login" for example should have role information to be used for scenario ID.

Persona:

Experienced Test Automation Architect who has 10 years of experience in test automation.

Output format:

1. Framework folder structure
2. Important components design
3. overall flow of execution using the components.
4. libraries to be used

Tone:

[STRICT] professional tone.

Question 1 of 5

Which test execution strategy should the framework primarily support?

- A. Sequential execution only
- B. Parallel execution at feature level
- C. Parallel execution at scenario level

- D. Distributed execution using Selenium Grid / Docker nodes
- E. Hybrid (local + Grid + cloud execution support)

E

Question 2 of 5

Which reporting and dashboarding solution should the framework support?

- A. Extent Reports only
- B. Allure Reports only
- C. Extent + Allure Reports
- D. Extent + ELK/Kibana Dashboard Integration
- E. Enterprise reporting with Extent + Allure + Jenkins trend analytics

A

Question 3 of 5

What type of test data management approach should the framework use?

- A. Static test data from Excel only
- B. Excel + JSON data files
- C. Excel + Database driven test data
- D. Excel + JSON + Environment specific property files
- E. Hybrid data strategy (Excel + JSON + DB + API-generated dynamic data)

D

Question 4 of 5

Which CI/CD execution strategy should the framework support in Jenkins?

- A. Manual Jenkins job execution only
- B. Scheduled nightly execution
- C. Git-triggered execution on every commit
- D. Multi-environment pipeline execution (QA/UAT/Prod-like)
- E. Full enterprise pipeline with parallel stages, tagging, rerun failures, and environment parameterization

E

Question 5 of 5

Which logging and framework observability strategy should be implemented?

- A. Basic console logging only
- B. Log4j logging with file appenders
- C. Log4j + screenshot capture on failure
- D. Centralized logging with execution tracing and API request/response logging
- E. Enterprise observability with Log4j, screenshots, API logs, execution tracing, retry analytics, and Jenkins artifact publishing

E

===== Output=====

Selenium + Rest Assured Automation Framework Architecture (Enterprise Grade)

1. Motivation for Automation

In a large enterprise environment, manual testing becomes a bottleneck due to:

- High regression cycles across multiple releases
- Multiple integrated systems (UI + APIs + backend services)
- Distributed teams across environments (QA/UAT/Prod-like)
- Frequent CI/CD deployments via Jenkins
- Need for faster feedback loops and higher release confidence

Automation Objectives:

- Reduce regression execution time significantly
- Ensure UI + API validation consistency
- Enable scalable parallel execution across environments
- Improve traceability of defects with logs, screenshots, and API traces
- Enable reusable, maintainable, and extensible test assets

How Architecture Achieves This:

- **Selenium + Page Object Model (POM)** → UI maintainability
- **Rest Assured** → API validation layer

- **Cucumber BDD** → Business-readable scenarios
 - **Maven** → Dependency & build management
 - **Jenkins Pipeline** → CI/CD integration with parameterized execution
 - **Hybrid Data Framework** → Excel + JSON + Properties (environment-based)
 - **Logging + Reporting + Observability Layer** → debugging + traceability
 - **Parallel execution + Grid support** → scalability
-

1. Framework Folder Structure

automation-framework/

```
|
|├── src/main/java/
| |├── config/
| | |├── ConfigReader.java
| | |├── EnvironmentManager.java
| | |
| |├── driver/
| | |├── DriverFactory.java
| | |├── DriverManager.java
| | |
| |├── pages/
| | |├── login/
| | | |├── LoginPage.java
| | |├── dashboard/
| | | |├── DashboardPage.java
| | |
| |├── api/
| | |├── base/
| | | |├── BaseAPI.java
| | |├── services/
| | | |├── LoginAPI.java
| | |
| |├── utils/
| | |├── ExcelUtil.java
```

```

├── JsonUtil.java
├── PropertyUtil.java
├── WaitUtil.java
├── ScreenshotUtil.java
├── LogUtil.java
├── hooks/
│   └── TestHooks.java
├── runners/
│   └── TestRunner.java
├── constants/
│   └── FrameworkConstants.java
├── src/test/java/
│   ├── stepdefinitions/
│   │   └── LoginSteps.java
│   ├── tests/
│   │   └── LoginTest.java (optional non-BDD layer)
│   └── src/test/resources/
│       ├── features/
│       │   └── login.feature
│       ├── testdata/
│       │   ├── login.xlsx
│       │   └── userData.json
│       ├── config/
│       │   ├── qa.properties
│       │   └── uat.properties
│       └── logs/
│       └── reports/

```

```
|  |— extent-report/  
|  |— allure-report/  
|  
|— pom.xml  
|— Jenkinsfile  
└— README.md
```

2. Important Components Design

A. Driver Layer (Selenium)

- DriverFactory
 - Creates WebDriver instance (Chrome/Firefox/Grid/Remote)
 - DriverManager
 - ThreadLocal driver management for parallel execution
-

B. Page Object Model (POM)

- Each page = separate class
 - Locators + actions encapsulated
 - Example:
 - LoginPage
 - enterUsername()
 - enterPassword()
 - clickLogin()
-

C. API Layer (Rest Assured)

- BaseAPI
 - Request specification builder
 - Authentication handling

- Service classes
 - LoginAPI, UserAPI, OrderAPI
-

D. Data Layer (Hybrid Model)

Supports:

- Excel (primary structured dataset)
- JSON (API payloads)
- Properties (environment configs)

Excel Design:

- Sheet: login
 - Columns:
 - TestCaseID
 - Role
 - Username
 - Password
 - ExpectedResult
-

E. Utility Layer

- ExcelUtil → read/write test data
 - JsonUtil → API payload handling
 - ScreenshotUtil → failure capture
 - LogUtil → Log4j logging
 - WaitUtil → explicit/fluent waits
-

F. Cucumber Layer (BDD)

- Feature files → business scenarios

- Step Definitions → glue code
 - Hooks:
 - BeforeScenario → setup driver/data
 - AfterScenario → teardown + reporting
-

G. Reporting Layer

- Extent Reports → primary execution report
 - Allure Reports → analytical reporting
 - Jenkins trend charts → historical execution tracking
-

H. Execution Layer

- TestRunner (Cucumber JUnit/TestNG)
 - Tag-based execution:
 - @smoke
 - @regression
 - Parallel execution enabled via TestNG
-

I. CI/CD Layer (Jenkins)

- Parameterized pipeline:
 - ENV (QA/UAT)
 - Browser
 - Tags
- Stages:
 - Checkout
 - Build (Maven clean install)
 - Execute tests

- Publish reports
 - Archive logs/screenshots
-

3. Overall Flow of Execution

Step 1: Jenkins Trigger

- Pipeline triggered manually or via Git commit

Step 2: Test Configuration Load

- Environment selected (QA/UAT)
- Property file loaded

Step 3: Driver Initialization

- DriverFactory creates browser instance
- DriverManager assigns thread-safe driver

Step 4: Feature Execution (Cucumber)

- TestRunner picks feature files based on tags

Step 5: Step Definition Execution

- Steps call:
 - Page Objects (UI)
 - API Services (Rest Assured)
 - Utility classes (data, logs)

Step 6: Data Fetching

- ExcelUtil reads dataset using TestCaseID
- JSON payload loaded if API test

Step 7: Validation Layer

- UI validations via Selenium assertions
- API validations via Rest Assured assertions

Step 8: Reporting & Logging

- Logs captured via Log4j
- Screenshots on failure
- Extent + Allure report generation

Step 9: Jenkins Publishing

- Reports archived
 - Trend charts updated
 - Email/Slack notifications triggered
-

4. Libraries to be Used

Core Testing

- Selenium WebDriver
- Rest Assured
- TestNG

BDD

- Cucumber Java
- Cucumber TestNG/JUnit

Build Tool

- Maven

Reporting

- ExtentReports
- Allure Reports

Logging

- Log4j2

Data Handling

- Apache POI (Excel)
- Jackson / Gson (JSON)

CI/CD

- Jenkins Pipeline
- Git integration

Utilities

- WebDriverManager
- Apache Commons IO

Parallel Execution

- TestNG Parallel Execution
 - Selenium Grid / Docker Grid
-

b) Test Strategy Document

Test Strategy Document - Prompt for step by step structured development of strategy

Prompt:

Test Strategy Document - Prompt - step by step /

Instructions:

Generate test strategy for testing credit card application and management for a bank

Ask 5 questions one after another and use each of previous inputs to proceed with next question and eventually the strategy development

[MANDATORY] sets organization standards, tools, automation approach

[STRICT] Practical and realistic approach.

Context:

Large banking firm needs a comprehensive test strategy to test credit application related end to end functionality.

Act as experienced credit card domain expert and test manager.

Examples:

Areas to be covered - regression, smoke, sanity, security (static and dynamic), stress/performance testing, usability testing.

Each driven by metrics to track and guide release over release like coverage, defect analysis

Persona:

Experienced program manager

Experienced test manager

Experienced test lead

Output format:

Scope, Motivation

Details of each area / aspect of testing section by section

Testing required for account, transaction, payment module, partners

Guardrails when using AI tools for helping analyze, conclude and drive testing.

Minimum criteria for release go-no-go.

Tone:

Professional tone and also mentoring tone for teams to follow.

We'll build this as a structured test strategy step-by-step. I'll ask one question at a time, and each answer will shape the final strategy document.

Question 1 of 5

What is the primary deployment model and release cadence for the credit card application system?

- A. Monolithic system with quarterly releases
- B. Microservices-based system with bi-weekly releases
- C. Hybrid (core banking monolith + modern digital microservices) with monthly releases
- D. Fully cloud-native microservices with weekly continuous deployment
- E. Legacy system with infrequent releases (quarterly or ad-hoc hotfix-driven)

C

Question 2 of 5

What are the primary customer interaction channels that must be covered in the testing strategy?

- A. Web portal only (desktop browser)
- B. Web + Mobile app only
- C. Web + Mobile app + Customer service (call center backend systems)
- D. Web + Mobile + Call center + Partner/bank branch assisted channels
- E. Omni-channel experience including Web, Mobile, Call center, Branch, Partner APIs, and ATM-linked services

B

Question 3 of 5

What is the expected risk and compliance posture for this credit card system (regulatory + security expectations)?

- A. Basic internal compliance checks only (low regulatory exposure)
- B. Standard banking compliance (PCI-DSS + internal audit controls)
- C. High regulatory compliance (PCI-DSS + SOX + regional banking regulations + audit readiness)
- D. Very high compliance (real-time fraud monitoring, AML integration, regulatory reporting automation)
- E. Critical financial system (full regulatory stack + continuous compliance monitoring + zero-trust security model)

Question 4 of 5

What should be the primary test automation and tool strategy for this credit card system?

- A. Manual testing-first with minimal automation (only smoke regression automated)
- B. UI automation focused (Selenium) with limited API automation (Rest Assured)
- C. Balanced automation (UI + API + DB validation) with CI/CD integration
- D. API-first automation strategy with UI as thin validation layer + strong data-driven testing
- E. Full automation-first strategy with UI, API, DB, performance automation + shift-left + shift-right testing integrated in CI/CD pipelines

(Your selection will define automation depth, tooling, and release readiness controls.)

Ads do not influence the answers you get from ChatGPT. Your chats stay private. Learn about ads and personalization

C

Question 5 of 5

What should be the primary performance and non-functional testing expectation for the credit card system?

- A. Basic load testing only before major releases
- B. Periodic performance testing (load + endurance) in QA environment
- C. Continuous performance testing integrated into CI/CD with defined SLAs (response time, throughput)
- D. Full non-functional suite including load, stress, spike, soak, resilience, and failover testing
- E. Enterprise-grade non-functional engineering with production-like simulations, observability, chaos testing, and real-time performance monitoring validation

(Your selection will define the depth of performance engineering and go-live readiness criteria.)

D

===== Output after question answers ===

Below is the **Enterprise Test Strategy Document** for a **Credit Card Application & Management System (Banking Domain)** based on your inputs.

TEST STRATEGY DOCUMENT

Credit Card Application & Lifecycle Management System

1. Scope & Motivation

1.1 Scope

This strategy covers end-to-end testing of the Credit Card system including:

Functional Scope

- Credit card application (new application, eligibility, approval workflow)
- Card lifecycle management (activation, blocking, replacement, renewal)
- Customer onboarding and KYC validation
- Payment processing and transaction authorization
- Billing, statements, and repayment processing
- Credit limit management and adjustments
- Fraud detection integration (basic validation layer)

Channels in Scope

- Web application (primary customer interface)
- Mobile application (secondary customer channel)

System Scope (Hybrid Architecture)

- Core banking monolithic system (credit decisioning, ledger)
 - Microservices layer (customer onboarding, notifications, card services)
-

1.2 Motivation

The credit card system is a **high-risk financial domain application**, where failures can lead to:

- Financial loss
- Regulatory non-compliance
- Customer trust damage
- Fraud exposure

Key Objectives:

- Ensure **zero-defect production release for financial-critical flows**
- Validate **end-to-end credit lifecycle accuracy**
- Ensure **regulatory compliance (PCI-DSS, banking audit requirements)**
- Improve **time-to-market via automation and CI/CD integration**

- Maintain **high system resilience under load and peak traffic conditions**
-

2. Test Approach & Strategy Areas

2.1 Functional Testing Strategy

Coverage Areas

- Credit application workflow validation
- Eligibility and credit scoring validation
- Card issuance and activation flow
- Transaction authorization and posting
- Payment processing (partial/full payments)
- Statement generation accuracy

Strategy

- Requirement-based test design
- End-to-end business flow validation
- Negative scenario coverage (declined applications, failed payments)
- Data-driven testing using customer profiles

Metrics

- Requirement coverage $\geq 95\%$
 - Defect leakage $< 2\%$ post-UAT
-

2.2 Regression Testing Strategy

Approach

- Automated regression suite covering:
 - Core application journey
 - Transaction processing

- Payment flows
- Executed on every build via CI/CD (Jenkins)

Scope

- Smoke subset for quick validation
- Full regression before major release

Metrics

- Regression automation coverage $\geq 80\%$
 - Execution time SLA ≤ 60 mins for smoke suite
-

2.3 Smoke & Sanity Testing

Smoke Testing

- Basic system availability checks:
 - Login
 - Application submission
 - Card status retrieval

Sanity Testing

- Focused validation after bug fixes:
 - Specific module verification
 - Payment correction scenarios
-

2.4 API & Integration Testing

Strategy

(API-first validation approach aligned with UI testing)

- Rest Assured based automation
- Validation of:
 - Credit scoring APIs

- Payment APIs
- Card lifecycle services
- Customer data services

Integration Points

- Core banking system
- Credit bureau systems
- Payment gateways
- Notification services (email/SMS)

Metrics

- API response SLA < 2 seconds (critical APIs)
 - 100% contract compliance validation
-

2.5 Security Testing Strategy

Static Security Testing (SAST)

- Code scanning for vulnerabilities
- Dependency vulnerability checks

Dynamic Security Testing (DAST)

- Runtime security validation:
 - Authentication bypass
 - Injection attacks (SQL/XSS)
 - Session management validation

Banking-Specific Checks

- PCI-DSS compliance validation
 - Data masking for sensitive information
 - Encryption validation (in transit + at rest)
-

2.6 Performance Testing Strategy

Approach (Enterprise-grade - D + E hybrid focus)

Includes:

- Load Testing (normal expected load)
- Stress Testing (peak beyond capacity)
- Spike Testing (sudden traffic bursts)
- Soak Testing (long duration stability)
- Resilience Testing (failure recovery behavior)

Key Scenarios

- Credit card application surge
- Payment processing peak loads (billing cycle)
- Transaction authorization spikes

Metrics

- Response time < 2–3 seconds (critical flows)
 - System uptime ≥ 99.9%
 - Zero critical memory leaks in soak testing
-

2.7 Usability Testing Strategy

Focus Areas

- Application form simplicity
- Mobile responsiveness
- Error message clarity
- Payment journey ease

Method

- UX walkthrough testing
- Heuristic evaluation

- User journey simulation
-

2.8 Data Testing Strategy

- Excel-driven datasets (role-based credit profiles)
 - JSON payloads for API testing
 - Environment-specific property files
 - Masked production-like datasets
-

3. Module-wise Testing Focus

3.1 Account Module

- Customer onboarding
- KYC validation
- Credit profile creation

3.2 Transaction Module

- Authorization
- Fraud checks
- Transaction posting

3.3 Payment Module

- Partial/full payment
- Payment reversal
- Late fee calculation

3.4 Partner Integrations

- Credit bureau validation
- Payment gateway integration
- Notification services

4. Tools & Organization Standards

Automation Tools

- Selenium WebDriver (UI)
- Rest Assured (API)
- TestNG (execution framework)
- Cucumber (BDD layer)

Performance

- JMeter / Gatling

Security

- OWASP ZAP (DAST)
- SonarQube (SAST)

Reporting

- Extent Reports
- Allure Reports

CI/CD

- Jenkins Pipeline (mandatory integration)
- Git-based triggers

Test Data

- Apache POI (Excel)
- JSON-based payload manager

Logging

- Log4j2

5. AI Usage Guardrails

AI tools may be used only under controlled conditions:

Allowed Use

- Test case generation support
- Log analysis summarization
- Defect clustering insights
- Requirement ambiguity clarification

Restrictions

- No direct AI-generated test execution decisions
- No AI-based release approval decisions
- No sensitive production data exposure to external AI tools

Governance

- Human validation required for:
 - Security findings
 - Release readiness decisions
 - Production incident analysis
-

6. Minimum Release Go/No-Go Criteria

A release is considered eligible only if:

Functional Criteria

- 100% critical test cases passed
- No open P1/P2 defects

Automation Criteria

- Smoke suite pass rate = 100%
- Regression pass rate $\geq$ 98%

Performance Criteria

- All SLAs met (response time, throughput)
- No critical degradation under load

Security Criteria

- No high/critical vulnerabilities open
- PCI-DSS compliance checks passed

Integration Criteria

- All dependent systems validated successfully
 - No failed API contracts
-

7. Summary

This strategy ensures:

- Strong **risk-based testing approach**
 - Balanced **UI + API automation coverage**
 - Enterprise-grade **performance + security validation**
 - Strong **CI/CD integration via Jenkins**
 - Clear **go/no-go governance model for banking releases**
-

If you want next, I can also produce:

- Detailed **test plan document (execution-ready)**
- **Test case design for credit card lifecycle**
- Or **Jenkins + automation framework integration architecture for this strategy**

1a) TL;DR

Too long ; Didn't read

Gave input TL;DR --- summarize both automation architecture and test strategy

Observations

This reduced earlier outputs to summary level bullets only.

output

TL;DR — Automation Architecture + Test Strategy (Credit Card Banking System)

1. Automation Framework Architecture (Selenium + Rest Assured + BDD)

Purpose

Enterprise-grade automation framework to support **UI + API testing**, scalable CI/CD execution, and maintainable test design for banking applications.

Core Design

Framework Stack

- **UI Automation:** Selenium WebDriver (Page Object Model)
 - **API Automation:** Rest Assured
 - **BDD Layer:** Cucumber
 - **Build Tool:** Maven
 - **Execution:** TestNG + Jenkins CI/CD
 - **Reporting:** Extent Reports + Allure Reports
 - **Logging:** Log4j2
 - **Data:** Excel + JSON + Environment properties (hybrid)
-

Architecture Highlights

Structure Principles

- Page Object Model for UI maintainability
 - Separate API service layer for reusable API calls
 - Data-driven testing using Excel (scenario IDs mapped to datasets)
 - Thread-safe execution via DriverManager (parallel runs)
 - Utility-driven framework (Excel, JSON, waits, screenshots, logs)
-

Execution Flow

1. Jenkins triggers pipeline (parametrized: env/browser/tags)
 2. Maven build + dependency resolution
 3. DriverFactory initializes browser (Grid/Local/Cloud)
 4. Cucumber runner executes feature files
 5. Step definitions invoke:
 - Page Objects (UI)
 - API Services (Rest Assured)
 - Data utilities (Excel/JSON)
 6. Assertions + validations
 7. Logging + screenshots on failure
 8. Reports published (Extent + Allure)
 9. Jenkins archives results + trend tracking
-

Key Capability Goals

- Parallel execution (Grid + local hybrid)
- Scalable CI/CD integration
- Reusable test components

- Strong traceability (logs + reports + screenshots)
 - Data-driven execution using Excel IDs
-

2. Test Strategy (Credit Card Banking System)

Purpose

Ensure **secure, compliant, high-quality credit card lifecycle system** across UI, API, integration, performance, and security layers.

System Context

- Hybrid architecture: Core banking + microservices
 - Channels: Web + Mobile
 - High-risk financial domain system
-

Testing Coverage

Functional Testing

- Credit card application workflow
 - Approval + credit scoring
 - Card lifecycle (activate/block/renew)
 - Payments + transactions + billing
-

Regression Strategy

- Automated CI/CD regression suite
 - Smoke + full regression cycles
 - High reuse across releases
-

API & Integration Testing

- Rest Assured-based automation
 - Validates:
 - Credit scoring APIs
 - Payment services
 - Core banking integration
 - Ensures contract consistency across systems
-

Security Testing

- SAST (static code analysis)
 - DAST (runtime vulnerability testing)
 - PCI-DSS compliance validation
 - Data encryption + masking validation
-

Performance Testing (Enterprise-grade)

- Load, stress, spike, soak, resilience testing
 - Validates:
 - Application surges (credit applications)
 - Payment peak loads
 - Ensures SLA compliance (response time, throughput, stability)
-

Usability Testing

- Mobile + web UX validation
 - Form simplicity + error handling + user journeys
-

Module Focus

- **Account:** onboarding, KYC, credit profile

- **Transaction:** authorization, fraud checks
 - **Payment:** full/partial payments, reversals
 - **Partners:** credit bureau + payment gateways + notifications
-

CI/CD Strategy

- Jenkins pipeline-driven execution
 - Environment-based testing (QA/UAT)
 - Tag-based test execution
 - Automated reporting + trend tracking
-

Quality Gates (Go/No-Go)

Functional

- 100% critical flows passed
- No P1/P2 defects

Automation

- Smoke: 100% pass
- Regression: $\geq 98\%$ pass rate

Performance

- SLA compliance met
- No critical degradation under load

Security

- No critical/high vulnerabilities
 - PCI-DSS compliance validated
-

AI Usage Controls

- Allowed: test design support, log analysis

- Not allowed: release decisions, production-sensitive data exposure
 - Human approval mandatory for go-live decisions
-

Final Summary

Automation Framework =

👉 Scalable **UI + API + BDD + CI/CD-driven architecture** with strong reporting, data-driven execution, and enterprise observability.

Test Strategy =

👉 Risk-based, banking-compliant, multi-layer testing approach covering **functional + API + security + performance + usability with strict release governance**.

If you want next level refinement, I can convert this into:

- Formal **enterprise test strategy PDF document**
- OR **interview-ready architecture diagram + explanation**
- OR **real implementation blueprint (code + Jenkins pipeline + folder skeleton)**

1b) ELI-15 Explain as if 15 years old is reading.

Input: ELI-15

Observations:

This gave very top level understanding in much simpler words.

Output:

Alright — here's the **ELI-15 (Explain Like I'm 15)** version of everything.

Big Picture

You are building and testing a **credit card system for a bank**, and you want:

- A **smart automation system** to test it fast and repeatedly
- A **testing strategy** to make sure the system is safe, correct, and fast
- A setup that works in **real enterprise banking environments**

Think of it like:

“A robot QA team + a strict safety checklist for a banking app”

1. Automation Framework (Simple Explanation)

What it is

A system that automatically tests the app instead of humans doing it manually.

How it is built

Think of it like LEGO blocks:

UI Testing (Selenium)

- Acts like a robot clicking buttons in web/mobile apps

API Testing (Rest Assured)

- Talks directly to backend services (faster than UI)

BDD (Cucumber)

- Test cases written like English sentences:
 - “Given user logs in... When user applies for card...”

Page Object Model (POM)

- Each screen = one reusable class
(Login page, Payment page, etc.)

Test Data

- Excel files = test inputs (like usernames, card types)
- JSON = API inputs

Jenkins (CI/CD)

- Runs tests automatically when code is updated

How it works (flow)

1. Jenkins starts the tests
2. Browser opens automatically
3. Tests run (UI + API)
4. Data is pulled from Excel
5. Results are checked
6. Report is generated

Goal of automation

- Run tests faster than humans
- Catch bugs early

- Run thousands of tests repeatedly without effort
-

2. Test Strategy (Simple Explanation)

What it is

A rulebook for how to test the credit card system properly

What we are testing

1. Functional Testing (Does it work?)

- Apply for credit card
 - Approve/Reject applications
 - Make payments
 - View statements
-

2. Regression Testing (Did we break something?)

- Re-run old tests after every change
 - Make sure nothing stopped working
-

3. API Testing (Backend checks)

- Check if services are working correctly
 - Example:
 - Credit score API
 - Payment API
-

4. Security Testing (Is it safe?)

- Prevent hacking
- Check:

- Login security
 - Data protection
 - Banking compliance (PCI-DSS)
-

5. Performance Testing (Can it handle load?)

We simulate:

- Many users applying at once
- Big payment spikes

Types:

- Load test → normal traffic
 - Stress test → extreme traffic
 - Spike test → sudden rush
 - Soak test → long hours running
-

6. Usability Testing (Is it easy to use?)

- Is app simple?
 - Are errors clear?
 - Can users understand flow easily?
-

Modules being tested

Accounts

- Customer signup
- KYC verification

Transactions

- Swipes, online payments
- Fraud checks

Payments

- Paying bills
- Partial/full payments

Partners

- Credit bureaus
 - Banks/payment gateways
-

Go / No-Go Rules (Release safety check)

Before releasing the system:

Must pass:

- All important features work
- No critical bugs
- Automation tests pass (almost all)

Performance:

- App is fast under heavy load

Security:

- No major vulnerabilities
-

AI Usage Rule (Important)

AI can help:

- Suggest test cases
- Analyze logs
- Find patterns in bugs

But:

- Humans must make final decisions
- AI cannot approve production releases

Final Simple Summary

Automation Framework =

👉 A robot system that runs UI + API tests automatically using Selenium + Rest Assured + Jenkins

Test Strategy =

👉 A strict plan that ensures:

- The credit card system works correctly
- Is secure like a bank should be
- Can handle heavy traffic
- Is easy for customers to use

One-line memory trick

👉 **Automation = “How we test automatically”**

👉 **Test Strategy = “What we must test to be safe in banking”**

If you want, I can next convert this into:

-  Simple diagram (architecture flow)
-  Interview-ready answer sheet
-  Real project folder + code starter kit